

REST and inter-process communication

Microservices represents the **domain-driven design** (known as **DDD**)-based domain service that runs as a process. Each microservice is independent. These independent services need communication with each other to implement the domain functionalities. No service directly accesses the database of other services. Instead, they use the APIs that are exposed by the service (microservice). These APIs could be implemented in various ways—using REST or events or gRPC. In this section, you'll learn how a service can consume the APIs of another exposed service using REST implementation.

Sample OTRS application services are registered and discoverable on `eureka-server`. Eureka Server allows the load balancing of calls using the Netflix Ribbon library. Spring Cloud also provides the discovery client. Remember that the `@EnableEurekaClient` or `@EnableDiscoveryClient` annotations are marked on the main application classes, which allow services to communicate to registration and discovery servers. `@EnableDiscoveryClient` is a generic discovery client and works with all registration and discovery server implementations, whereas `@EnableEurekaClient` is more specific to the Netflix Eureka service. These clients allow us to search and call the services discoverable using registration and discovery server.

Using REST clients, you can call other service REST endpoints to communicate, which we'll discuss in the next section. These REST clients need a hostname and port number. Looking at the dynamic nature of deployment infrastructure and avoiding hardcoding, we'll use the discovery and registration server. This means that we just need the service IDs instead of hostnames and ports.

Spring Cloud provides `DiscoveryClient`, which communicates with the registration and discovery server. If you want a Netflix Eureka-specific client, then you can use `EurekaClient`. Have a look at the following code:

```

@Component
class DiscoveryClientSample implements CommandLineRunner {

    private static final Logger LOG =
        LoggerFactory.getLogger(DiscoveryClientSample.class);

    @Autowired
    private DiscoveryClient discoveryClient;

    @Override
    public void run(String... strings) throws Exception {
        final String serviceName = "restaurant-service";
        // print the Discovery Client Description
        LOG.info("\n{}", discoveryClient.description());
        // Get restaurant-service instances and prints its info
        discoveryClient.getInstances(serviceName)
            .forEach(serviceInstance -> {
                LOG.info("\nInstance --> {} \nServer: {} \nPort: {} \nURI: {} ",
                    serviceInstance.getServiceId(),
                    serviceInstance.getHost(), serviceInstance.getPort(),
                    serviceInstance.getUri());
            });
    }
}

```

The output shows two instances, as shown in the following code:

```

Composite Discovery Client
Instance: RESTAURANT-SERVICE
Server: SOUSHARM-IN
Port: 3402
URI: http://SOUSHARM-IN:3402

Instance --> RESTAURANT-SERVICE
Server: SOUSHARM-IN
Port: 368
URI: http://SOUSHARM-IN:3368

```

You may want to remove the port setting from the YAML

configuration to use the dynamic port by the service for running multiple instances.

This code is self-explanatory. It prints the restaurant service instance details. I would suggest that you execute the multiple instances of `restaurant-service` and then check the output of this code. This code is placed in the main class of `booking-service`. It prints the details of each of the `restaurant-service` instances. You can place this code in any service, or can change the service ID and check the results.

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)